

VERIQO

Evidence-Qualified Intelligence OS — Enterprise Architecture and Assurance Report

Repository: `veriqo` (Go 1.24.7) · **Branch:** `claude/veriqo-enterprise-architecture-7j56b9`

Report date: 5 September 2026 · **Round 6** (supersedes Rounds 1–5)

Status: SPECIFICATIONALLY IMPLEMENTED. NOT PRODUCTION QUALIFIED.

Round 6 posture: the assurance kernel is CLOSED. Layers 0–3 of the assurance ladder are complete and further self-verification is refused by `pkg/assurance/boundary`. What remains is not more engineering; it is five battlefields and thirteen gates that require a party who is not VERIQO.

That wording is deliberate and replaces "engineering complete", which an investor can read as a conclusion and stop. Twenty gates block release; thirteen need a party that is not VERIQO. Nothing in the assurance register is above `INTERNALLY_ASSURED`.

0. What changed in Round 6 — the round that stops

Round 6 is not another round of hardening. The audit that opened it made one finding that overrides the rest, and acting on it meant doing less, not more.

The finding: assurance recursion

Every layer of assurance in this repository was added for a reason that was good at the time. The invariant chokepoint caught a real class of defect. The assurance mutation suite attacked the assurance graph and found something. The verifier-of-the-verifier found a bug.

Each step was justified by the step before it. That is exactly the shape of a runaway:

```
system
-> assurance of the system
  -> assurance of the assurance
    -> verifier of the assurance
      -> test of the verifier
        -> test of the test
          -> ...
```

There is no natural stopping point, because at every rung the next rung can be justified by the same argument that justified the current one. A system that follows it becomes **a machine for continuously verifying its own verification machinery**, and value creation stops — not with a failure, but with a green board and nothing shipped.

The audit's judgement was that Round 5 must be the last round of recursive assurance architecture. That judgement is correct, and Round 6 implements it.

1. The boundary — pkg/assurance/boundary

```
0 SYSTEM
1 ASSURANCE
2 ASSURANCE_OF_ASSURANCE
3 VERIFIER_OF_ASSURANCE      <- VERIQO's ceiling; all four CLOSED
                             THE BOUNDARY
4 INDEPENDENT_VERIFICATION   <- a different party
5 EXTERNAL_ASSESSMENT        <- a party with standing
```

`Propose()` classifies any piece of work. Above the boundary it is **REFUSED**, and the refusal names the party who must do it instead. At a closed kernel layer it is **REDUNDANT** — not forbidden, which matters: the boundary is not a prohibition on engineering — unless it is a direct dependency of an external gate, in which case it is allowed, because the constraint is what the work *unblocks*, not which layer it sits on.

A good rationale buys nothing. Every rung on a runaway ladder has a good rationale; that is what makes it a runaway. So the rationale is recorded and deliberately not weighed, and a test proves four separate good arguments — "a customer asked", "it would take two days", "the last three layers each found a defect", "it is a dependency of G4" — all fail to get past the line.

The uncomfortable part, stated in the package's own documentation rather than hidden: **this package is itself layer 3**. The argument for adding it is the same argument that justifies every other rung. The only difference is that it terminates the ladder instead of extending it. If a future round cites it as precedent for a layer 4, it has failed.

2. The Qualification-Driven Engineering Freeze — pkg/freeze

Not a freeze on development. A freeze on **discretionary scope**: work enters only as a direct dependency of a *named* external gate or a customer pilot. Correctness fixes are always permitted, because a freeze that forbids fixing a bug will be ignored, and a rule that is ignored constrains nothing.

Judgement is not the constraint, because judgement is what produced the runaway — each decision looked like the responsible one. So the rule is external to the judgement: name the gate, or do not enter.

Round 6's own work is the first thing weighed by it. **Eight items permitted, two refused:**

Refused	Why
a verifier for <code>veriqo-verify</code>	layer 4 — the exact runaway. A verifier VERIQO writes to check the verifier VERIQO wrote shares every assumption of what it checks
a generalised temporal-reasoning engine	the most interesting engineering left in the system, and no external gate is waiting on it

A freeze register containing only approvals is a record of a freeze that was not applied. Two tests enforce this: one fails if Round 6 refused itself nothing, and one fails if nothing in the refused list is described as *worth doing* — refusing only bad ideas is not a constraint.

3. The headline that VERIQO cannot move — pkg/dashboard

The headline was "**37 passed, 0 failed**". True, cheap to raise, and an answer to a question nobody outside engineering asked. Worse, it moves: a team managing to that board adds tests, while the figures that matter stay exactly where they are.

It is now banned by name, with its reason, alongside six other figures that rise when VERIQO works harder and cannot fall when VERIQO becomes less trustworthy. The nine measures that replace it:

Measure	Reads	Scope
Critical controls externally validated	0 / 20	the mandatory production gates
Real-world corpus coverage	NOT MEASURED	no partner has defined the population
Independent assessments completed	0 / 5	pentest, crypto, redaction lab, counsel, model eval
Production operational evidence	0 / 30 days	longest run to date: seconds, one process, one host
Replay determinism	1 / 1 (<i>internal</i>)	one synthetic case, in-process
Unauthorised authority attempts blocked	NOT MEASURED	no deployment, so no denominator
Cross-tenant isolation violations	0 / 7 (<i>internal</i>)	VERIQO's own attacks, which is a statement about the attacks
Evidence provenance completeness	1 / 1 (<i>internal</i>)	the synthetic case, where VERIQO built the custody records
Disproof routes available	1 / 1 (<i>internal</i>)	findings VERIQO also designed the route for

NOT MEASURED renders differently from zero, and a test enforces it. Zero is a measurement — we looked and found none. **NOT MEASURED** is the absence of one — there is no denominator, because the thing that would supply it does not exist. Rendering them alike would be the reporting layer committing the exact error the epistemic firewall exists to prevent.

A test requires that **nothing on the board is movable by VERIQO alone**.

4. When the world is ugly — pkg/intel/maritime

Round 5 was strong on synthetic correctness. The real question is different: *can VERIQO stay epistemically honest when the data is a mess?*

A 40-knot jump between two AIS reports is consistent with **five** mechanisms: GNSS spoofing, GNSS interference, a failing receiver, a relay that stamped its own arrival time onto the position, and a message that simply arrived late and out of order. Every one of them is common. GNSS interference in particular is now routine in several sea areas — reporting through 2025 described large and rising numbers of vessels affected near specific coastlines,

though *I have not independently verified those counts and would treat the specific figures as needing a primary source.*

The point does not depend on the figures. A system that prints `SPOOFING` will be confidently wrong in public, on a data partner's own data, in front of the people who know that sea area best.

`Explain()` returns every consistent cause with the modality that would discriminate it — and **no discriminator lives inside AIS**. A single feed cannot separate a spoofed position from a jammed one, because in both cases the receiver reports a position it believes. Separating them needs satellite imagery, terrestrial RF, a port record, a receiver log or a feed audit.

Three structural refusals:

- `Triage` has no `MostLikely`, no `Verdict` and no `Score`, and a test asserts those methods **do not exist** — so a caller in a hurry cannot find one.
- Mundane causes sort first. A list led by `SPOOFING` teaches the analyst what to look for.
- `GENUINE_MOVEMENT` is always on the list. A detector whose explanations omit "the thing happened" has assumed its conclusion before looking.

5. Corroboration that is really repetition — `pkg/qualification/independence`

Fifty sources report the same fact; forty-nine are copies of the fiftieth. The corroboration count says fifty and the finding is founded. This is the classic route by which a false report acquires apparent verification.

The existing graph handles copies that *declare* their origin. That is the easy half. This handles the normal case, which declares nothing — using the truth-discovery principle that **sources sharing unusual errors are unlikely to be independent**. Two analysts can independently reach the right answer; they do not independently make the same transposition.

Agreement with the majority is explicitly **not** an indicator. That is what independent sources do, and treating it as dependence would collapse every honest corroboration.

One choice worth recording, because the obvious option was wrong. The text measure is **containment** — shared tokens over the *shorter* account — not Jaccard. Jaccard penalises length difference, and the commonest real copy is a wire story reprinted with a paragraph added. On the flagship case that reprint scores **0.84 on Jaccard and 1.00 on containment**; a Jaccard threshold low enough to catch it would also catch unrelated accounts of the same event. A minimum-length guard then stops containment firing on short generic sentences, where a "copy" finding would be a finding about the English language.

There is no function that reports two sources as independent, and a test asserts there is none. Finding no shared error establishes that no shared error was found. An undeclared common upstream looks exactly like independence. Demotion only — `UNKNOWN ≠ NEGATIVE`, applied to the dimension where the temptation to promote is strongest.

6. The Decision Passport as the product — `pkg/passport/commercial`

Nobody purchases an epistemic firewall, an assurance graph, engine separation or a mutation suite. What a customer purchases is the answer to one question:

"Show me why you reached this conclusion, what evidence supports it, how independent those sources are, what you rejected, and exactly what would prove it wrong."

That is the Decision Passport. Everything else in this repository is machinery for making it truthful.

Fourteen sections, in the order the audit specified — and the order is an argument:

- **observations before hypotheses**, because a reader given the conclusion first reads the evidence as support for it;
- **contradictions before hypotheses**, not in an appendix — a passport that buries its conflicts is a sales document;
- **disproof before replay**, because how to attack this matters more than how to re-run it;
- **external status last**, so `NOT_EXTERNALLY_QUALIFIED` is what the reader leaves with.

`Validate()` refuses: a percentage as a confidence state (the number a reader multiplies by an exposure and puts in a reserve calculation); VERIQO or any automated principal as the decision authority; a single hypothesis; an empty `UNKNOWN` column; an absent disproof route; absent limitations; and an observation citing a source not in the provenance table — an assertion placed in the observations section being the most effective place in the document to hide one.

7. One case, worked end to end — `pkg/intel/maritime/flagship`

```
vessel -> cargo -> voyage -> port event -> AIS -> weather -> document
-> contradiction -> source independence -> hypotheses
-> decision -> passport -> disproof route
```

A cargo-quantity dispute: a six-hour AIS reporting gap in the Singapore Strait, a 6.3 m draught increase across it, and a bill of lading stating the full contracted quantity was loaded the previous day at the nominated port.

It is **synthetic in every particular**, and the notice saying so sits above the case header rather than in a footnote.

It is built to come out **ambiguous**, deliberately. A worked example that resolves cleanly demonstrates the machinery on the one input shape that never occurs, and teaches the reader that VERIQO produces answers. The demonstration worth having is one where the evidence is genuinely insufficient, the system says so, and it is *still useful* — because it names the six things that would settle it and what each would cost.

The case carries the classic ship-to-ship transfer signature precisely because that is what a system is most likely to over-call. Its conclusion is `INSUFFICIENT_TO_DECIDE`, its five reporting accounts reduce to three observations under copy detection, and its decision authority is a named human being.

8. The five battlefields — pkg/readiness

Each states its **PASS and its FAIL condition in advance**. A pass condition alone is a target: without a matching fail condition, a disappointing result gets reinterpreted afterwards as a partial success.

#	Battlefield	Blocked by	Passes when
1	Real maritime data	B-AIS	every anomaly raised carries a triage naming what would settle it
2	Real documents	B-CORPUS	zero leakage over several hundred documents with real variation
3	Independent attack	B-PENTEST , B-REDTEAM	no critical finding, and low overlap with VERIQO's own suite
4	Real dispute case, read blind	—	readers reach their own conclusion and find the same decisive uncertainty
5	Operational endurance	B-INFRA , B-SOAK	30 days with failure injection and zero replay divergence

Battlefield 3's second condition is the subtle one: high overlap with the internal suite would mean the engagement reproduced VERIQO's own assumptions rather than testing where VERIQO cannot see.

Battlefield 4 is the one that **cannot be discharged by receiving a document**, and its pass condition is deliberately *not* agreement with VERIQO. A blind reader who agrees for reasons the passport does not contain has been **persuaded rather than informed** — which is the failure the passport format exists to prevent.

What Round 6 did not change

The status. Twenty gates block release; thirteen need a party that is not VERIQO. Nothing is above `INTERNALLY_ASSURED`. Eleven evidence debts, every one requiring an outside party. No party outside VERIQO has read any claim in this repository.

`scripts/verify.sh` reports **50 passed, up from 37** — and that figure is now explicitly banned from the headline board. The board that matters reads four zeroes, two `NOT MEASURED`, and three figures marked internal-scope-only.

That is the honest state of a system that has finished building itself and has not yet been outside.

0a. What changed in Round 5

The Round 5 audit made six findings. None was a missing feature; all six were places where the honest machinery still permitted a dishonest reading.

#	Finding	The failure it names
1	The causal chain is not explicit	observation → interpretation → hypothesis → assertion → decision collapsed into one word, "finding"
2	Blockers are not a schedule	"needs an independent assessor" cannot be bought; a line item with a validator, a deliverable, a dependency, a lead time and a cost class can
3	Three registers, one missing	Engineering and external qualification had boards; the epistemic dimension — the actual product — had none
4	Challengeability is documentary	CHALLENGE.txt invited attack and gave the attacker nothing to run
5	Mutation testing reads as proof	It is evidence of robustness under a mutation operator set; it is not proof that no defect exists
6	Four engines, drawn not enforced	OBSERVE / ARBITRATE / QUALIFY → DECIDE was a diagram, and a diagram cannot be violated

1. The epistemic ladder — pkg/epistemic/ladder

The chain is now five typed rungs, and each one may only rest on specific rungs below it:

OBSERVATION	rests on nothing. It is where evidence enters, and it carries who recorded it and which evidence version.
INFERENCE	rests on OBSERVATION or INFERENCE, and names its method.
HYPOTHESIS	rests on anything up to HYPOTHESIS, and must name at least one competing alternative and a discriminator.
ASSERTION	rests only on HYPOTHESIS, and names who stands behind it.
DECISION	rests only on ASSERTION — and VERIQO may never record one.

The rules that matter are the refusals:

- **An INFERENCE may not rest on a HYPOTHESIS.** Doing so launders a guess into a derivation, and every conclusion drawn from it inherits the confidence of the derivation rather than of the guess.
- **A HYPOTHESIS with no alternatives is an ASSERTION in disguise**, and is refused as one. A hypothesis that competes with nothing was never tested.
- **VERIQO may not record a DECISION** (`ladder.ErrNotOurs`). It assembles one.

`JudgementDistance(id)` counts the rungs between a statement and the observations under it — the number of places confidence compounds. It is not a score and is not combined with anything.

Every refusal explains what it prevents rather than restating the rule, because a reader who is told the rule argues with the rule.

2. The procurement graph — pkg/readiness

Round 4's readiness dimensions named *who* was blocking. That is still an answer nobody can act on. Round 5 turns the same blockers into fifteen line items, each carrying a validator type,

the deliverable that party must hand back, what must be true before it can start, a lead time and a cost class:

```
CRITICAL PATH (4 step(s))
B-FREEZE -> B-INFRA -> B-SOAK -> B-RELEASE
```

`StartableNow()` lists what can be begun today; `ByValidator()` groups the work by who must be engaged, so one conversation covers several blockers. A test requires every blocker to name a validator qualification specific enough to write a purchase order against — "an accredited penetration testing firm with cryptographic review capability" passes; "a security expert" does not.

This does not make VERIQO closer to production. It makes the remaining distance priceable, which is the difference between a gap and a plan.

3. Three boards, and the one that was missing — `pkg/assurance/metrics`

The registers are renamed and restructured:

Board	What it measures	Who can move it
<code>ENGINEERING_INTEGRITY</code>	does the code do what it says	VERIQO
<code>EPISTEMIC_INTEGRITY</code>	does the system reason honestly	VERIQO
<code>EXTERNAL_QUALIFICATION</code>	has anybody outside checked	nobody inside VERIQO

The middle board did not exist, which meant the property VERIQO actually sells — disciplined reasoning under uncertainty — was measured nowhere. It now carries eight measures: refusal coverage, contradiction retention, unparseable-not-absent handling, ladder conformance, alternative-hypothesis density, and so on.

The third board is **empty**, and the panel says so in those words rather than rendering it as zero. Zero is a measurement; empty is an absence of one, and the difference is the whole system.

The boards are never summed. `SelfProducible()` returns false for exactly one of them, and a test pins which.

4. Executable challengeability — `pkg/assurance/capsule`

`CHALLENGE.txt` invited attack. An auditor who accepted the invitation had to invent their own attacks, which meant most of them would not.

The capsule now ships `challenge/kit.txt`, a protocol with five phases — RUN, VERIFY, ATTACK, COMPARE, REPORT — and three things stated **in advance**:

- **six expected outputs** (X-01...X-06), so a run that differs is a finding rather than a discussion;
- **seven negative cases** (N-01...N-07) — a truncated bundle, an edited claimed qualification, a re-signed passport — each with the exact refusal the system must produce;

- **seven known failure modes** (K-01...K-07) that are wrong and **not fixed**, published so that finding one is not presented as a discovery.

And the sentence that makes the kit honest: a negative result means *these attacks did not succeed under these conditions*. It does not mean the system is secure.

5. The mutation suite says what it does not cover

`test/assurancemutation` now states, as a test rather than a comment, that surviving its mutations is evidence of robustness under a specific operator set and is **not** proof that no defect exists. The operators it does not apply — different defaults, different migration state, different database state, different deployment configuration — are named.

A suite that reported only its kills would be the artefact this system exists to refuse.

6. The four engines, made refusable — `pkg/engine`

The audit's own diagram:



It was right, and it was a drawing. `pkg/engine` makes each separation refusable, and each refusal names the failure it prevents:

- **OBSERVE may not rank.** An observation engine that picks the right reading destroys the losing reading before anybody sees it.
- **ARBITRATE may not grade its sources.** Merged with QUALIFY, a strong argument from a weak source outranks a weak argument from a strong one, and nothing says so.
- **QUALIFY may not fetch.** An engine that may collect what it needs will collect what confirms the grade it has already formed.
- **DECIDE may not be closed by VERIQO.**

ARBITRATE and QUALIFY are **siblings, not a sequence**. Ordering them either way lets one see the other's answer before forming its own, and both directions are a way of grading the conclusion you wanted. A test runs the passage in both orders and requires both to work.

Two refusals are not in the diagram and should be:

- **An engine may not be re-run in place.** A second ARBITRATE that silently replaced the first would let an unwelcome result be re-run until it came out differently, with no trace

that it ever came out the other way.

- **A stage that ran and produced nothing is not a stage that did not run.** The first is a finding about the evidence; the second is a hole in the process. A type that rendered them alike would present the hole as a finding.

`Seal()` requires a replay manifest **and** a disproof route. The last row of the diagram is a condition on the passport existing, not a feature to be added later: a decision that cannot be re-run and cannot be shown wrong is a document with a signature on it.

`pkg/engine` and `pkg/epistemic/ladder` state the same rule at two scales. The ladder refuses a DECISION rung recorded by VERIQO for one chain of reasoning; `pkg/engine` refuses a DECIDE stage closed by any principal whose *kind* is automated, for the system. Neither depends on the other, which is deliberate: the rule holds twice.

7. Every package now carries its own tests

A sweep found four packages with no direct test file — covered only incidentally, through callers, so a defect in them surfaced somewhere else or not at all.

- `pkg/entity` — `Identifier.Matches` is the silent-merge site. MMSI is reassigned on reflagging, so two vessels eight years apart carry one number, and a match that ignores the validity interval fuses them permanently and unrecoverably. The tests pin the temporal-overlap requirement and the half-open interval semantics.
- `pkg/evidence/redaction/worker` — direct tests of the refusals rather than the happy path, plus the property that verification searches the *inspectable view* rather than the raw compressed bytes.
- `cmd/veriqctl` — the report table and the print order were locals inside `main()`, so nothing could check that they agree. A name in one and not the other made `veriqctl all` call a nil function, and that had already happened once. They are now package-level and held together by a test.
- `cmd/veriqo-verify` — its two loaders are the whole of its out-of-band trust input. An **empty key file is refused**, because loading it silently would fall back to the bundle's own keys while the operator believes they checked authenticity. An **empty revocation list is not refused**, because "I looked and nothing is revoked" is a finding, not an absence. An **unparseable revocation list is refused** rather than read as empty — that is `UNPARSEABLE` $\neq$ `ABSENT`, committed by the verifier itself.

What Round 5 did not change

The status. Twenty gates still block release. Nothing is above `INTERNALLY_ASSURED`. No party outside VERIQO has read any claim in this repository. Round 5 made the system harder to misread; it did not make it qualified, and no amount of Go will.

0b. What changed in Round 4

The Round 3 audit was titled *Epistemic Firewall*, and it named the deepest principle in the system so far — plus five ways the honest machinery could still mislead.

#	Finding	The failure it names
1	The Epistemic Firewall	unknown → assumed → scored → trusted → decision, where nothing is ever falsified
2	H1-H4 as pseudo-certification	"4/5 — nearly certified". H5 is not the next checkbox
3	No assurance mutation testing	Nothing attacked the assurance graph <i>itself</i>
4	A readiness <i>level</i> hides <i>who</i> is blocking	"How much further?" is the wrong question when the answer is "somebody else"
5	Source confidence as one number	A legality objection offset by a fresh timestamp
6	Assurance bureaucracy risk	More registers, more ledgers — and no investigation moving faster

0.1 The Epistemic Firewall

Unreadable evidence can never increase assurance.

Four refusals, stated as inequalities because each names a pair that a system, left alone, will eventually treat as identical:

UNREADABLE	!= VERIFIED	a document nobody could parse has not been checked and found clean -- it has not been checked
UNPARSEABLE	!= ABSENT	a field that failed to decode is a FAULT in the observation; one that is not there is a FACT
MISSING	!= VALID	skipping a check is not passing it
UNKNOWN	!= NEGATIVE	not having found something is not having found its absence

Every one of these failures happens through a **zero value**: an unpopulated field is empty, empty compares equal to "nothing was there", and that is routinely read as "nothing is wrong". So `pkg/epistemic` makes the unreadable case a *value* rather than an absence, and the zero value is `UNEXAMINED` — not "fine", and not "absent" either, because **nobody looked, and that is a third thing**.

Two consequences worth stating. `ABSENT` deliberately cannot increase assurance: confirmed absence is real information *about what is not there*. And coverage reports four counts, **never a ratio** — "4 of 6" reads as two-thirds of the way to something, and the missing two are not two-thirds of anything: one of them may be the one that mattered.

0.2 H5 is not the next checkbox

```
H1 [x]  H2 [x]  H3 [x]  H4 [x]  H5 [ ]
```

Every human being reads that as *"four of five — nearly certified"*. The reading is not careless; it is **what a five-item checklist means**. And it is wrong:

```
H1..H4  VERIQO evaluating VERIQO
H5      an INDEPENDENT PARTY evaluating VERIQO
```

That is not one more check. It is a **change of who is speaking**, and no number of internal checks moves any distance toward it. So the levels are grouped by epistemic source and never counted:

```
INTERNAL CLAIM SCREENING  H1 H2 H3 H4
EXTERNAL CLAIM VALIDATION NOT PERFORMED. No check in this group exists.
```

There is no `Fraction`, no `Score`, no `Percent` and no `N/M` anywhere in the package — and a test asserts the report contains none. It ends: *"Everything above is VERIQO evaluating VERIQO. It is not four fifths of anything."*

0.3 Assurance mutation testing

`test/adversarial` assumes somebody who wants to change what VERIQO says about the **world**. `test/assurancemutation` assumes the likelier attacker: somebody who wants to change what VERIQO says about **itself** — not a hostile outsider, an insider under commercial pressure three days before a deadline, editing a field.

Nine targets, every one rejected:

Mutation	Rejected because
Validator changed to VERIQO	The validator must not be the implementer
Independence claimed without attestation	<code>AttestedBy</code> cannot be empty or self
Internal evidence relabelled external	The class check runs on the validator
Artefact hash removed	A report could be reused for a later version
Claim level raised	Unsupported by evidence of the required class
Open counterexample hidden by raising the level	Caps at <code>IMPLEMENTED</code>
Qualification emitted above the evidence	Corrected downward at every surface
Gate stripped of its controls	Refused at construction
Every gate's required level lowered	Release still refused — the debts block it independently

That last row is the structural result: if lowering the bar closed a gate, the bar would be the only thing holding it — **and a bar is a number in a file**.

A mutation suite asserts something a test suite cannot: that each *specific field* is load-bearing. A field nobody checks can be changed freely, and no test that only exercises valid records will ever notice.

0.4 A status that names a party

Dimension	Status	Blocked on
Architecture	INTERNALLY_ASSURED	—
Semantics	INTERNALLY_ASSURED	—
Implementation	INTERNALLY_ASSURED	—
Security	PENDING_EXTERNAL	An independent assessor
Cryptography	PENDING_EXTERNAL	An independent assessor
Legal	PENDING_COUNSEL	Counsel, per jurisdiction
Data rights	PENDING_PARTNER	A commercial partner
Operations	NOT_YET_PROVEN	Nobody — infrastructure and time
Production	NOT_QUALIFIED	Every dimension above

A *level* invites "how much further?" — the right question for work the builder can do and the wrong one for everything else. A **status names the party**. Nine dimensions rather than five, because collapsing security with cryptography, or data rights with legal, hides that they are blocked on **different parties with different lead times**: one row says *find an assessor*; two say *find an assessor and a cryptographer*, which is a different procurement.

The report's most useful section is now **WHO WE NEED**, and it ends with: *nothing remaining is movable by the builder alone*.

0.5 The Source Trust Vector

```
source confidence = 0.8
```

is the most damaging simplification available in this domain. A source whose **ATTRIBUTION** is unknown and whose **AUTHENTICITY** is confirmed is not "somewhat trustworthy" — it is **a document we can prove is genuine and cannot say who wrote**. That is a specific situation with specific consequences, and 0.8 erases it.

Worse, the number is directional in a way its users forget: a weakness in **LEGALITY** cannot be compensated by strength in **TIMELINESS**. Averaging them produces a figure in which **a lawyer's objection is offset by a fresh timestamp**.

Nine dimensions, no combining function. Only `LEGALITY` is disqualifying — everything else weakens, and weakness is for a human to weigh. `Weakest()` returns a **dimension**, because the answer to "how much can I trust this" is a name.

0.6 Track B: the maritime chain, and an answer that is not a finding

The audit's sharpest strategic warning: VERIQO must not become an *assurance bureaucracy machine* — more registers, more ledgers, more manifests, and no investigation moving faster.

```
INTELLIGENCE OS      TRUST OS
What happened?       Can we prove it?
What is happening?   Why should we trust it?
What may happen?     What contradicts it?
      \               /
      DECISION PASSPORT
```

`test/maritime` runs one vertical end to end: observation → evidence → provenance → independence → contradiction → hypothesis → **decision passport** → replay. The case has the shape real ones have — two AIS vendors reselling one network are **one producer**; an anonymous tip resolves to **nobody** and makes the whole structure `UNASSESSABLE`.

And the result is deliberately **not a finding**:

```
H1  0.38  cargo was loaded during the gap
H2  0.34  ballast was taken on and the earlier draught was stale
H3  0.28  the draught values are a data-entry artefact
x H4  0.00 the vessel left and returned
      -- under 4 NM either side of the gap; no departure could produce that

NO HYPOTHESIS IS MEANINGFULLY AHEAD.
```

That is the answer real evidence gives most of the time, and **a pipeline that can only produce conclusions is one that will produce them when it should not.**

0.7 The Decision Passport, and the ordering rule

A dashboard says "*here is the answer*". A passport says: here is the answer, the evidence, the contradictions, the provenance, the uncertainty, and exactly what would overturn it.

The unflattering numbers come first. Contradictions and unresolved questions render *above* the hypotheses; qualification and external validation are the last thing a reader sees. Ordering is not presentation here — a document that leads with its conclusion and buries its contradictions is **making an argument**, and this is not supposed to be one.

The header counts cannot drift from the lists: a summary that disagrees with its own body is how a document becomes a misrepresentation without anybody lying.

0.8 The External Challenge Package

The capsule is no longer assembled to *prove VERIQO is safe*. It is assembled to **make it easy for an outsider to prove VERIQO wrong** — almost the same files, a completely different document, and only the second is worth an assessor's time.

CHALLENGE.txt ranks our own weakest points, by name:

1. **The canonicaliser** — every digest passes through it; a divergence would be invisible from inside (ED-011)
2. **The verifier's shared code** — if you do not supply your own canonicaliser, our verifier's PASS is worth less than it looks
3. **Redaction irrecoverability** — we claim absence in twelve encodings, not irrecoverability, and nobody has ever tried (ED-004)
4. **The injection defence** — ten tests, no counterexamples, written by the people who wrote the defence (ED-005)
5. **The assurance layer itself** — nine mutation targets; find a tenth
6. **The source-class lawfulness model** — engineering's reading, not counsel's (ED-010)

It also says what would *not* be a useful finding: *that the system is not production ready. It says so itself, in code, on every run.*

0c. What changed in Round 3

The Round 2 audit found something more uncomfortable than a missing feature: **the system's own honesty machinery could produce false assurance**. Six findings, all now closed.

#	Finding	What it would have caused
1	Test inflation — 814 tests as a quality proxy	The cheapest way to raise quality becomes writing tests that assert what the code already does
2	The honesty checker is itself a keyword screen	An overclaim detector that overclaims — with a green tick attached
3	Law 11 was a package property, not a system invariant	Any other surface writing "PRODUCTION_QUALIFIED" bypasses it, and will not look like a bypass
4	Source <i>counting</i> instead of producer attribution	Four outlets carrying one wire story counted as four sources
5	"The full chain verifies end to end"	A bundle verification carried away as a platform qualification
6	A control with no claim was a missing row, not a finding	Work presumed fine, invisible to every report of what is unproven

0.1 Test inflation, and three registers that cannot be combined

"814 tests" is impressive and it is a trap.

One independent penetration test is worth more than two hundred additional unit tests. One real-world corpus is worth more than a hundred synthetic fixtures. Those are **not comparisons between bigger and smaller numbers** — they are comparisons between

different *kinds* of evidence, and a dashboard showing both invites the arithmetic that destroys the distinction.

SOFTWARE VERIFICATION tests, coverage, race, vet, fuzz, mutation	ASSURANCE QUALIFICATION external evidence, independent validation, real corpus, legal review	PRODUCTION EVIDENCE uptime, incidents, recovery, rotation, failover, access review
the builder can move this alone	only somebody else can move this	only running it can move this

`pkg/assurance/metrics` has **no function that takes two registers** and no aggregate of any kind. VERIQO's own panel: eight measures in the first, each carrying what it does *not* show; the other two **EMPTY**. They are constructed with no measures rather than with zero values, because a row reading "independent assessments: 0" invites a reader to see a scale with a low value on it, when what exists is **no scale**.

0.2 The honesty checker that overclaims

```
honesty checker -> FALSE PASS -> false assurance
```

If a check that detects overclaim is itself a keyword screen, the system manufactures exactly the confidence it exists to refuse — with *more* credibility than an unchecked claim, because it now carries a tick.

Level	Name	Asks	Defeated by
H1	CLAIM_LANGUAGE_SCREENING	Does the text contain phrases we decided not to use?	Paraphrase
H2	STRUCTURAL_CLAIM_ANALYSIS	Does the artefact have the parts it needs?	Filling fields with nothing
H3	SEMANTIC_CONTRADICTION_ANALYSIS	Does it contradict itself?	Being consistent and wrong
H4	EVIDENCE_TO_CLAIM_VALIDATION	Does the evidence support the claim?	Evidence that is itself mistaken
H5	INDEPENDENT_EXTERNAL_REVIEW	Did somebody who is not the author read it?	A reviewer sharing the author's assumptions

`DescribeSafely` refuses the phrase "honesty verification" at any level below H5, at the point the description is written. Every level states what defeats it — **including H5**, because a reader told what defeats a check cannot mistake it for one nothing defeats.

VERIQO's own grading is uncomfortable, which is the point. The passport conclusion screen is H1. The suite tops out at H4. **H5 is not performed at all.** `verify.sh`'s section was renamed from "honesty checks" to "overclaim checks" for the same reason.

`Highest()` is deliberately not an average: forty H1 checks reach H1, and averaging would let quantity substitute for strength — the test-inflation failure in a different costume.

0.3 Law 11 as a system-wide invariant

No system surface may emit an assurance state higher than the state derived from the evidence.

A package refusing self-certification constrains one code path. A system has many surfaces that can utter an assurance state — release authority, passport issuer, API, CLI, every report, the qualification ledger, the capsule, any export, automation, CI, and a UI nobody has written yet. If one of them can write `"PRODUCTION_QUALIFIED"`, Law 11 is bypassed — **and the bypass will not look like a bypass. It will look like a field being set**, in a package whose author had no idea Law 11 existed.

So `pkg/assurance/invariant` is the one chokepoint, and an **architecture test parses every file outside the assurance layer and fails on a string literal naming a high state**. The scanner has its own test proving it would catch a synthetic bypass, and another proving the exemption list has not grown to cover the module.

`Emit` never returns an error for an over-claim. A surface receiving an error would have to decide what to do, and some would log it and publish anyway. Instead the over-claim is *impossible*: the caller receives the derived state whatever it asked for, plus a record of what it asked for.

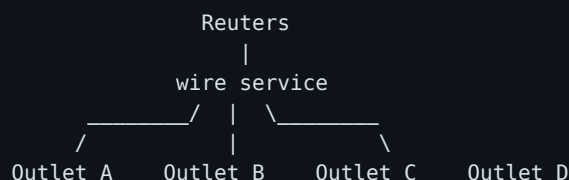
The capsule now proves this on itself. It asks for `PRODUCTION_QUALIFIED` deliberately — asking for what we expect to get would leave the invariant untested at the one place it matters — and publishes what comes back:

```
claimed: PRODUCTION_QUALIFIED
derived: INTERNALLY_ASSURED
verdict: QUALIFICATION_CLAIM_INVALID
emitted: INTERNALLY_ASSURED
```

CLAIMED ≠ DERIVED is frozen as a type. A claim below the evidence is *not* promoted either — a party may claim less than it could, and quietly upgrading them would make this package the thing it prevents.

0.4 The Source Independence Graph

Pairwise assessment answers "are these two independent". That is the right question and not the whole one, because independence is a property of the **producer structure**:



observed sources = 5
independent producers = 1

The harder case is three anonymous posts. **Both tempting answers are wrong:** 3 treats "we do not know" as "they differ"; 1 asserts they are the same, which is equally unfounded. So `UNASSESSABLE` is its own value — *not a low count, the absence of one* — and `SatisfiesCorroboration` returns false for it at every threshold.

One unattributable source contaminates the whole answer: a structure with two known producers and one unknown does **not** satisfy a requirement for two, because the third could be either of them.

0.5 Capsule verification ≠ platform qualification

Round 2's report said "the full chain now verifies end to end". True of that chain, and one sentence away from a serious misreading. Every verification result now carries, **on pass and on failure alike**:

```
capsule chain: verifiable, by anyone, in milliseconds
VERIQ0:       NOT EXTERNALLY QUALIFIED
```

It is printed every time rather than once in a preface, because the reader who most needs it is the one who has stopped reading prefaces.

0.6 The Disproof Route

A disproof *path* is a sentence describing a destination. A recipient in a dispute needs the **route**: numbered steps, each naming a party who can take it, what it produces, and what happens to the finding if it succeeds — ordered cheapest-first so somebody who runs out of budget has still done the most informative thing available.

A blocked step **stays in the route**: a recipient is entitled to know the cheapest refutation is closed to them. A route with every step blocked says so loudly — an unchallengeable finding is a limitation, not a strength. And `IfAllFail` is required, because a route describing only refutation implies that surviving it *proves* the claim.

0.7 The verifier-of-verifier, and what it found

Every other test asks whether the verifier catches tampering. These ask whether it is **capable of catching anything**. For each step there must exist an input that makes that step FAIL — a step for which none exists is indistinguishable from one that always passes.

It found a defect immediately: **the canonicalize step silently skipped any .json file that failed to parse**. A corrupted document passed verification *by being unreadable* — the worst

possible direction for that error to run.

0.8 Assurance Orphans

A control nothing claims anything about never appears in a report of what is unproven, because nothing was ever claimed. It is invisible in exactly the way that matters. `ASSURANCE_ORPHAN` is now a typed finding carrying the gates resting on it, where to look, and its consequence.

A good assurance system should generate uncomfortable findings. If every audit returns GREEN, the likeliest explanation is that the audit is bad.

0d. What changed in Round 2

Round 1 built the qualification kernel and reported honestly on it. The audit that followed made a sharper point, and it was correct:

VERIQO could constrain what it said about the world far better than what it said about itself. An assurance position lived as prose and a gate lived as a status field, so "tested by its author" and "attacked by an accredited third party" occupied the same green cell.

Round 2 closes that. Five things are new:

#	What	Why it matters
1	Law 11 — No Self-Certification	An assurance level requiring independent evidence is now <i>unrepresentable</i> without it
2	The Master Assurance Graph	Two registers became one walkable structure: gate → control → claim → evidence → validator → level → release
3	The Independent Verification Kit	A third party can check VERIQO without asking VERIQO anything
4	The Auditor Capsule	"Run it yourself" replaces "we say so" — and it now carries a case that can actually be run
5	The Intelligence Fabric	Source classes with lawfulness constraints, and vessel-behaviour analysis, so the kernel is not the whole product

Plus: readiness as five dimensions instead of a percentage; the gate lifecycle as a state machine; `ESTIMATE ≠ MEASURED ≠ VALIDATED ≠ PRODUCTION_PROVEN` as a type; the decision passport as six product kinds; and the repository hygiene the audit flagged.

65 packages · 168 files · ~54,000 lines · 918 tests · `go vet` clean · `verify.sh` 31/31.

And the number above is exactly the kind of figure §0.1 exists to warn you about.

1. Law 11, and why it is the most important thing here

An entity may implement and test a control, but may not unilaterally promote that control to an assurance level whose definition requires independent evidence.

This is the assurance-layer twin of Law 7. Law 7 says an AI cannot upgrade evidence. Law 11 says an author cannot upgrade the assurance of their own control. Both are enforced the same way: the promotion is not discouraged, it is **unrepresentable**.

The ladder

```
UNDEFINED
-> SPECIFIED
-> IMPLEMENTED
-> INTERNALLY_TESTED
-> INTERNALLY_ASSURED      <- the last rung VERIQO can reach alone
-----
-> READY_FOR_EXTERNAL_TEST
-> EXTERNALLY_TESTED      <- needs a party that is not the implementer
-> EXTERNALLY_VALIDATED
-> OPERATIONALLY_PROVEN   <- needs a production deployment
-> PRODUCTION_QUALIFIED   <- needs the release authority
```

No state jumps. A promotion from `INTERNALLY_ASSURED` to `PRODUCTION_QUALIFIED` is refused, and the refusal *names the rungs it skipped* — because every one of them is a question nobody answered: was it tested by somebody else, did that find anything, was it fixed, was the fix retested, has it run, for how long, under what load.

Demotion is deliberately cheap. It may skip any distance, needs no evidence, and needs only a reason. Making the honest move expensive is how organisations end up holding stale assurance.

The three attacks it refuses

1. **Internal evidence for an external rung.** Refused on evidence class.
2. **Evidence labelled external, produced by the implementer.** Refused: the validator must not be the implementer.
3. **A validator attesting to its own independence.** Refused: `AttestedBy` cannot equal the validator's own id.

2. The Master Assurance Graph

Gates, controls and assurance claims were three lists. They are now one graph, and a release decision walks it to the bottom:

```
GATE —> CONTROL —> ASSURANCE CLAIM —> EVIDENCE —> VALIDATOR —> LEVEL —> RELEASE
```

Every hop is checked. A dangling reference fails at construction. And the graph reports the quiet failure a checklist cannot see: **a control that no claim says anything about** — work that exists, is presumed fine, and has no stated property anybody could test.

What it says about VERIQO, in code rather than prose

Fact	Enforced by
Nothing is above INTERNALLY_ASSURED	TestNothingInTheRegisterIsAboveInternallyAssured
No gate is closable	TestNoGateIsClosable
All 20 mandatory gates rest entirely on VERIQO's own evidence	TestEveryMandatoryGateRestsOnVeriqosOwnEvidence
Every control has at least one claim	TestEveryControlIsClaimedAbout
Every unmet claim names an evidence debt with an owner	TestEveryClaimNamesADebtOrIsFullySupported

Each of those is a test that **fails when it stops being true**, which is the only way a statement like this stays honest for longer than one release.

Evidence Debt

"OPEN" tells a reader something is unfinished. It does not tell them what is missing, why, who would supply it, what it costs, what it blocks, or the consequence of shipping without it. Those six answers are the difference between a gap a buyer can *price* and one that reads as vagueness — and vagueness makes an honest position look evasive.

Eleven open debts. All eleven require a party that is not VERIQO.

ID	Missing	Needs
ED-001	Independent security assessment	An accredited pentest firm
ED-002	A production key root	An HSM or cloud KMS
ED-003	An external anchor for checkpoints	A timestamping authority
ED-004	Real documents, and a recovery attempt	A corpus supplier and an adversarial lab
ED-005	Independent test of the injection defence	A red team with AI-agent experience
ED-006	Operational evidence	Production infrastructure
ED-007	A live data contract and a real case	A commercial data provider
ED-008	SBOM, signing, vulnerability feed	A signing authority
ED-009	An evaluation set VERIQO did not build	An outside party
ED-010	Legal opinion on source-class lawfulness	Counsel, per jurisdiction
ED-011	Cross-implementation canonicaliser conformance	An independent RFC 8785 implementation

ED-011 was found by the register's own tests: the canonicalisation claim was short of its required level and had no debt behind it, so the gap had no owner. Every digest, ledger record, passport and replay comparison passes through that one function, and a divergence from the standard would be **invisible from inside** — the system would be perfectly self-consistent and silently unable to interoperate with anything else.

3. A gate is a state transition, not a checkbox

```
OPEN -> READY -> TESTING -> FINDINGS -> REMEDIATED -> RETESTED -> VALIDATED
      ^^^^^      ^^^^^^      ^^^^^^^^^^      ^^^^^^
      VERIQO    needs an    VERIQO's    needs the
      alone    outside party own work    outside party again
```

`OPEN` does not reach `VALIDATED`, however the caller spells it. The move that makes every assurance programme worthless —

```
status: OPEN    ->    status: CLOSED
```

— is refused by the type system, and the refusal names the path it skipped.

Three further rules, each closing a way the lifecycle could be gamed:

- **"Nothing was found" must be recorded explicitly.** An absent finding record and a finding-free one are the two situations a green row conflates.
 - **An accepted risk needs a rationale.** It is a decision, not a fix, and the two must not look alike.
 - **A gate cannot be `VALIDATED` with an open finding.**
-

4. The Independent Verification Kit

The verifier must not trust the system being verified.

That rules out the obvious design. A verifier that asks VERIQO for a status has not verified, it has **relied**. One that compares a digest to a digest VERIQO also computed has confirmed only that VERIQO is self-consistent — which a compromised system also is.

`cmd/veriqo-verify` takes a bundle and **recomputes**:

Step	What is recomputed, and why it matters
Canonicalize	Every JSON document canonicalises to a fixed point
Artefact hashes	Digests come from the bytes , not from the records
Signature	The payload digest is recomputed before the signature is checked — a verifier checking against the <i>supplied</i> digest would accept any payload at all
Provenance	Every record reaches a named origin with an ordered hop path
Ledger lineage	Rehashed from genesis — a chain carries its own hashes, and reading them confirms only that the file describes itself
Replay	Deterministic steps re-executed; RECORDED steps reported as not re-executed
Revocation	"Not checked" and "not revoked" are different answers
Qualification state	Derived , then compared with the claim

That last row is what makes it a verifier. A bundle asserting `PRODUCTION_QUALIFIED` on internal evidence is **contradicted, not believed**, and the report says: *"THESE DISAGREE. Believe the derived value."*

What it says it cannot establish, on every run

1. **Key authenticity.** A bundle produced entirely by an impostor is internally perfect. Key trust is out-of-band.
2. **Existence in time.** Without an external anchor, a hash chain proves only its own consistency, so a wholesale rewrite between two observations is invisible.
3. **Anything about evidence left out.**
4. **A defect in the canonicaliser** — when the default is used, it is the *same code* the system used, so both sides make the same mistake and agree. The `Canonicalizer` seam exists so a verifier can supply their own, and the report names which was used.

`UNVERIFIABLE` is a first-class outcome, distinct from `FAIL`. Reporting "cannot check" as "invalid" trains readers to ignore failures; reporting it as a pass is a lie.

5. The Auditor Capsule

What an assessor is handed so they can say *"show me"* instead of reading a document: the assurance register, the gates, the failure classes, the self-doubt register, the policy rules and authority matrix as data, the API contracts, the redaction corpus with both figures and their epistemic status, a dependency manifest, a build manifest, a threat model — **and a worked case the verifier can actually run.**

```
go run ./cmd/veriqoctl capsule ./capsule
go run ./cmd/veriqo-verify ./capsule      # every step PASSes
```

Three deliberate properties:

- **It claims exactly** `INTERNALLY_ASSURED`. Claiming less than the evidence supports costs nothing. Claiming more costs the engagement.
- **The build manifest does NOT claim verified reproducibility**, because no bit-for-bit comparison has been done. Gate G19.
- **The threat model's** `not_considered` **list is a required field**. A threat model that lists only what was thought about reads as complete. Ours excludes, among others: toolchain compromise, side channels, post-quantum adversaries, and a compromised operator with both commit access and release authority acting deliberately over time.

The worked case, and its most dangerous property

Without one, six of the verifier's eight steps report `UNVERIFIABLE` and the assessor is back to reading a document. With one, everything passes — **and that is the risk**. An assessor who watches every step pass can carry away the impression that VERIQO has been shown to work on data. It has been shown to work on data VERIQO wrote for the purpose.

So the capsule says the case is synthetic in four places, and its README says:

What it establishes about VERIQO's behaviour on REAL data: nothing. Every byte of it was written by VERIQO to be verified by you. That is evidence debt ED-007.

The replay record deliberately contains one `DETERMINISTIC` and one `RECORDED` step. A capsule with only deterministic steps would misrepresent what replay establishes in a real case.

6. Four things a percentage sign hides

```
ESTIMATE ≠ MEASURED ≠ VALIDATED ≠ PRODUCTION_PROVEN
```

These are not confidence levels. They name **where a number came from**, which is why they cannot be interconverted: no amount of re-running an estimate makes it a measurement, and no amount of internal measurement makes it validated.

`pkg/assurance/epistemic` makes the bare form unavailable — a `Figure` renders with its status or not at all — and a derived figure inherits the **weakest** status of its inputs. A conclusion drawn from an estimate and a measurement is an estimate.

VERIQO's own worked example: **88% weighted redaction coverage is an ESTIMATE**, because the prevalence weights are judgements and the corpus is VERIQO's own.

7. Readiness without a number

"VERIQO is 85% production ready" is a sentence with no truth conditions. It cannot be checked, cannot be disagreed with in detail, and invites the reader to interpolate — 85% sounds like six weeks. The reality it usually describes is a system whose architecture is finished and whose external validation has not started: two facts no single number can carry.

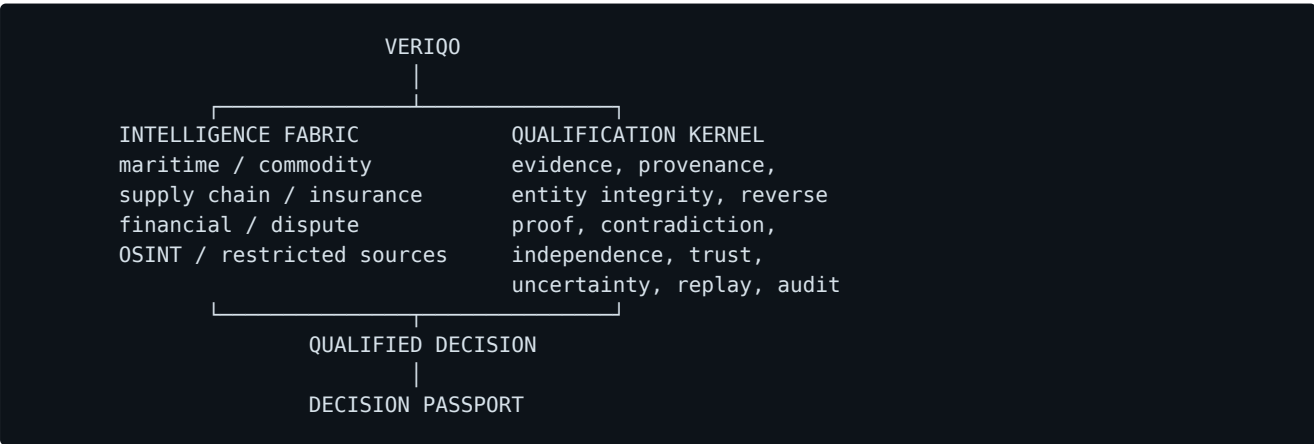
Dimension	Level	Who can move it
Architecture	HIGH	The builder
Semantics	HIGH	The builder
Implementation	SUBSTANTIAL	The builder
Production infrastructure	NOT_STARTED	Requires a deployment
External validation	NOT_STARTED	Requires an outside party

The asymmetry is the point: effort moves the first three and **cannot move the last two at all**. A dimension needing an outside party cannot be recorded above `NOT_STARTED` by an internal assessor — "we believe we would pass a pentest" is a hope, not an assessment, and a model that recorded it as one would defeat its own purpose.

There is deliberately no `Overall()`. The summary sentence is **generated from the assessments**, so it cannot drift away from them.

8. The Intelligence Fabric

The audit warned that VERIQO must not collapse into an evidence-governance system. The kernel is the moat; domain intelligence is the market surface. Neither is the product alone.



8.1 Source classes — including the restricted ones

Real intelligence platforms touch material ranging from a public register to a leaked corpus circulating on a hidden service. Pretending otherwise produces a system that is either useless

or quietly unlawful.

The framing that matters: that range is not a quality gradient. Treating it as one loses the two things that actually decide the outcome:

- **LAWFULNESS.** A breach corpus is not low-quality. It may be unlawful to hold, and no amount of corroboration changes that.
- **ATTRIBUTION.** An anonymous paste has no producer, so Law 2 cannot be satisfied and Law 6 cannot be evaluated — two anonymous pastes might be one person.

Eleven classes, with enforced constraints. The consequential ones:

Class	May found a finding	Counts for corroboration	Permitted uses
OFFICIAL_REGISTER	yes	yes	screen, lead, corroborate, found, disclose
ADVERSE_MEDIA	no	no	screen, lead
ANONYMOUS_DISCLOSURE	no	no	screen, lead
HIDDEN_SERVICE_FORUM	no	no	screen, lead
BREACH_DERIVED	no	no	screen only

Numerousness does not substitute. Ten anonymous disclosures are still zero producers. Six outlets carrying one wire story are one source — which is the commonest way a screening system manufactures confidence.

`BREACH_DERIVED` and `HIDDEN_SERVICE_FORUM` are deliberately **separate** classes. They are routinely conflated and are not the same thing: a forum post about a vessel's movements is not stolen data, and a breach corpus on the public web is. The venue affects attribution; the acquisition affects lawfulness.

Restricted classes cannot be held **at all** without a recorded legal basis naming a jurisdiction, a purpose, and counsel. Engineering's reading of the law is not an opinion — hence ED-010.

This package acquires nothing. There is no connector, crawler, credential or address in it. It is a classification and a set of refusals — the part of handling such material that belongs in code. Acquisition is a legal and operational decision made by people with names.

8.2 Vessel behaviour

Eight detectors over position reports: great-circle distance and bearing, implausible speed, reporting gaps, null positions, identifier collision, rendezvous, draught change, and reported-versus-implied speed disagreement.

Every detector answers a narrow question and refuses the broad one. It reports *"these two reports are separated by a distance no hull could cover"*, never *"this vessel spoofed its position"* — the same observation is produced by a receiver clock error, a transposed digit, two

vessels sharing an identifier, and a falsification, and choosing between those is a claim about intent.

So every anomaly carries its **innocent explanations**, ordered most-mundane-first, and a **diagnostic**: what evidence would separate them. An anomaly without those is an accusation wearing the clothes of a measurement, and an analyst who sees a hundred of them learns to ignore all of them.

Three details that decide whether such a detector is usable at all:

1. **Position error is subtracted before implied speed is computed.** Two reports ten seconds apart, each accurate to half a nautical mile, imply 360 knots with the vessel stationary.
2. **0,0 is treated as the garbage value it almost always is** — an uninitialised field, a failed parse — not as a position in the Gulf of Guinea.
3. **A gap is stated as a fact about the RECORD.** "The vessel went dark" is a claim about the vessel *and about intent*. On a coastal-only network, the report leads with the receiver: offshore silence is the expected behaviour of the *receiver*.

9. The decision passport as a product

Nobody buys an answer. An answer from a system is worth what the system's reputation is worth, and in a dispute that is nothing — the other side's expert has an answer too. What a customer buys is a package that survives being attacked by somebody paid to attack it.

Six kinds, differing in **what they are forbidden to say**:

Kind	May not say	Because the decision belongs to
Claim qualification	"is covered", "is fraudulent"	The insurer's claims handler
Incident evidence	"was at fault", "caused the"	The investigating authority or tribunal
Quantity discrepancy	"was stolen", "the seller is liable"	The parties, or the tribunal
Collateral evidence	"is good security", "title is good"	The bank's credit committee
Dispute evidence	"the claimant will succeed"	The tribunal
Counterparty qualification	"is sanctioned", "is high risk"	The firm's compliance officer

Each forbidden statement is one the customer in that domain **will ask for and must not be given**. A system that says them is not neutral, and a neutral system that says them once is not neutral any more.

Two defects the tests found in the first version of this screen, both the classic failure of keyword filtering: inflection evaded it ("are good security" vs "is good security"), and an explicit disclaimer tripped it — flagging *"whether the policy responds is not addressed here"* would have taught authors to **delete their disclaimers**, the precise inversion of the purpose.

The check is documented as a screen, not a proof. A determined author can convey "covered" without writing the word. It makes the accidental case impossible; the real control is that one person mints and another approves.

10. The adversarial suite, and what it found

43 tests that assume an attacker. Each asserts the *structural* reason an attack cannot work, not that it happened not to.

Three real defects, found on the suite's first run and all now fixed:

Defect	Consequence before the fix
A ledger checksum failure <i>anywhere</i> was treated as a torn tail	Editing record 2 of 4 silently discarded records 2, 3 and 4 and reopened the chain at height 2, with no error
The same path on a front-truncated log	The chain opened as empty
Every decompressed read used <code>io.LimitReader</code>	A part inflating to 256 MiB was truncated to 64 MiB, redacted, released and marked Verified — 192 MiB absent and never searched

Two of the three were in exactly the controls a marketing claim would rest on: the immutable ledger and the verified derivative.

Stated carefully: they were found by the party that wrote them, and these attacks were designed by somebody who knew where the code looked. That is the strongest available evidence that the disproof paths are real, and it is *not the same as being attacked*. Gates G15-G18 exist for that reason, and ED-005 owns it.

The three levels of adversarial testing

```
A1 developer adversarial testing      <- VERIQ0 is here
A2 independent grey-box testing
A3 black-box real-world testing
```

11. The self-doubt register

Seventeen claims, each with a proof path **and** a disproof path — not a negative test, but an attempt to produce a counterexample. Outcomes are `ESTABLISHED`, `REFUTED`, or `UNSETTLED` (the proof succeeded and the disproof was not run — *not* established).

The register now also records **closed counterexamples**: a defect the disproof path did produce and that has since been fixed, with the fix cited.

"We attacked this and found nothing" and "we attacked this, found a defect, and closed it" are different pieces of evidence, and the second is much stronger. A claim that has never yielded may mean the control is sound or may mean the attack was weak. A claim whose attack once succeeded is one whose attack is **known to be capable of succeeding**.

A test asserts the register as a whole: *if not one disproof path had ever produced a counterexample, either the system is perfect or the attacks are not real, and the second is far more likely*.

Deliberately **UNSETTLED** :

- **CLAIM-REDACTION-IRREVERSIBLE** — absence in twelve encodings is not irrecoverability, and nothing here attempts recovery.
- **CLAIM-INJECTION-STRUCTURALLY-REFUSED** — ten adversarial tests attack it and none has produced a counterexample, which is *precisely the weaker position*.

Every claim in this register was attacked by VERIQO and by nobody else.

12. Article 18 — redaction coverage

23 structural variants: 17 accepted, 6 refused by design, 0 failed.

- **Structural coverage: 74%** — the share of *variants*, **not** a pass rate. Every refusal is safe; none is a capability failure.
- **Weighted coverage: 88% (ESTIMATE)** — weighted by a *judgement* of how common each structure is.

Three of the six refused variants are COMMON in real documents — **PDF-ENCRYPTED** , **PDF-INCREMENTAL** , **PPTX-EMBEDDED-IMAGE** . A real population would land there in bulk and **the 88% would fall**.

Corpus qualification level: **L2_FIXTURE_CORRECTNESS** . Every fixture was built by VERIQO. Nobody has attempted to recover redacted content. *Surviving one's own corpus is the weakest form of survival available*.

The rule the worker turns on: **anything it cannot decode is refused, never reported clean**. A document nobody could read is a document nobody can certify as redacted.

13. The twenty gates and the nine-dimension scorecard

All twenty gates blocking. Thirteen require a party that is not VERIQO.

Dimension	Rating
Evidence integrity	YELLOW
Entity integrity	YELLOW
Reasoning integrity	YELLOW
Security	RED
Operational reliability	RED
Data rights	YELLOW
AI governance	YELLOW
Replayability	YELLOW
External validation	RED

Nothing is GREEN. Release refused, with nine reasons. There is no aggregate score, deliberately: a single figure lets a strong dimension carry a weak one, and the weak one is what a customer needs to see.

`EXTERNAL_VALIDATION`'s RED bounds every other rating. *No party outside VERIQO has examined, attacked, validated or corroborated any part of this system.*

14. What is deliberately absent

- **An aggregate confidence score** — nine uncertainty dimensions, no `Overall()`.
- **A readiness percentage** — five dimensions, no aggregate.
- **A `CONFIRMED` verdict** — evidence can fail to disconfirm; it cannot confirm.
- **A merge operation** — five outcomes, and a merge needs a reviewer.
- **A trust path from source to conclusion.**
- **A production KMS** — `SoftwareRoot` is a test double and refuses production mode.
- **An anchor implementation** — an anchor VERIQO controls proves that VERIQO agrees with itself. The interface is declared and left unimplemented, and the register records the absence as a *decision* so it does not read as an oversight.
- **`time.Now()` on any deterministic path** — enforced by an architecture test.
- **Any acquisition of restricted-class material** — no connector, no crawler, no credential.

15. Verification

```
./scripts/verify.sh    # 50 passed, 0 failed, 16 explicitly NOT run
```

The honesty checks fail the build if the system starts overstating itself:

- the scorecard refuses its own release
- no gate is satisfied
- **nothing is above** INTERNALLY_ASSURED
- **every mandatory gate rests on VERIQO alone**
- coverage carries the word ESTIMATE
- readiness offers no aggregate figure
- **every readiness status names its blocking party**, and nothing remaining is movable by the builder alone
- every evidence debt has an owner *and* a risk
- **veriqo-verify passes every step on a freshly built capsule** — and fails the build if any step is unverifiable
- **capsule verification is not platform qualification** — the scope statement must appear
- **no overclaim check is described above its level**, and the suite does not reach H5
- **the three boards are not combined**, and the empty one is called out
- **the four engines are separated** — DECIDE is closed by a human principal only
- **a decision that cannot be re-run or disproved is refused** by Seal()
- **the epistemic ladder separates seeing from concluding**
- **the procurement graph names a critical path**
- **the assurance ladder has a stated boundary**, and a fifth layer of self-verification is refused
- **the freeze refused something Round 6 wanted**
- **the test count is not the headline**, and NOT MEASURED is not rendered as zero
- **nothing on the headline is movable by VERIQO alone**
- **an AIS gap is never called spoofing**, and no discriminator for it lives inside AIS
- **fifty copies of one story are one observation**
- **the flagship passport refuses to conclude**, says it is synthetic first, and cuts five accounts to three observations
- **every battlefield states a fail condition**

Sixteen things it explicitly does **not** run, each tied to a gate or a debt — including independent canonicaliser (ED-011), external anchor check (ED-003), and independent red team (G15-G18), and H5 external review of claims — no party outside VERIQO has read any claim in this repository.

16. The lifecycle, restated

The old model was BUILD → TEST → CLOSE GAPS → RELEASE. It is wrong in a way that matters: it treats gaps as things the builder closes, and most of the remaining ones are not.

```
BUILD -> TEST -> COUNTEREXAMPLE -> FIX -> INTERNALLY ASSURE
-> PACKAGE EVIDENCE -> INDEPENDENTLY VERIFY
-> OPERATE -> MEASURE -> EXTERNALLY QUALIFY -> PRODUCTION PROVE
-> CONTINUOUSLY REQUALIFY
```

VERIQO has reached PACKAGE EVIDENCE. The next step needs somebody else.

And the rule underneath all of it: process success is not evidence of real-world success. A green verify.sh means the code builds, the tests hold, and the system still refuses to overstate itself. It does not mean the system works on anything real, and this report is written so that no reader can conclude otherwise.

17. Do not chase GREEN

The audit's closing point, and the right one to end on.

VERIQO must not be built to produce GREEN. It must be built so that GREEN appears **only when it is earned** — because in maritime dispute, commodity, insurance and trade finance, **one false green can be worth millions**, and a correct RED is worth far more than a wrong GREEN.

Everything in this report is downstream of that. The refusals, the four inequalities, the mutation suite, the status that names a party, the passport that leads with its contradictions — none of them make the system look better. Every one of them makes it harder for the system to look better than it is.

18. The honest summary

Architecture high, semantics high, implementation substantial. Production infrastructure not started, external validation not started. The weakest dimension is production infrastructure, and no single figure is offered because a strong dimension must not be allowed to carry a weak one.

Or, for a customer asking whether this can be trusted:

VERIQO's core qualification kernel is internally assured. Production deployment remains gated by independently verifiable security, operational, data-rights and external-validation evidence — eleven named debts, every one of which requires a party that is not VERIQO.

The engineering problem is largely closed. The **qualification** problem is now the bottleneck, and it cannot be closed by writing more Go. That is not a failure of this report; it is its principal finding — and a platform that reports it accurately about itself is the only kind that can be trusted to report it about anything else.

Every figure regenerable: `go run ./cmd/verioctl all`. Every claim checkable without trusting this document: `go run ./cmd/verioctl capsule ./c && go run ./cmd/veriqo-verify ./c`.